Fixed-Income RFQ Trading Platform — Legacy-to-AWS Migration Architecture Package

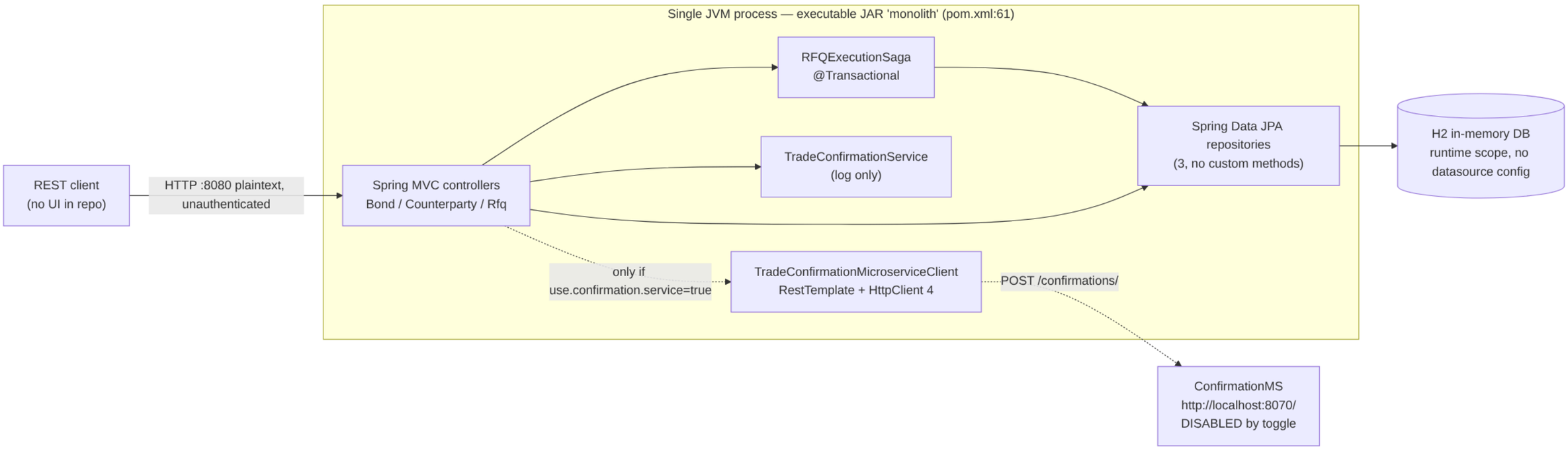
Repository `COG-GTM/Fixed-Income-RFQ-Trading-Platform`, commit `0b72d81`. Contents: current state, AWS target state (including the assumed service list, the assumed guardrails in force and the guardrail-compliance table), open questions, and the migration plan ending at production cutover.

Current State — Fixed-Income RFQ Trading Platform

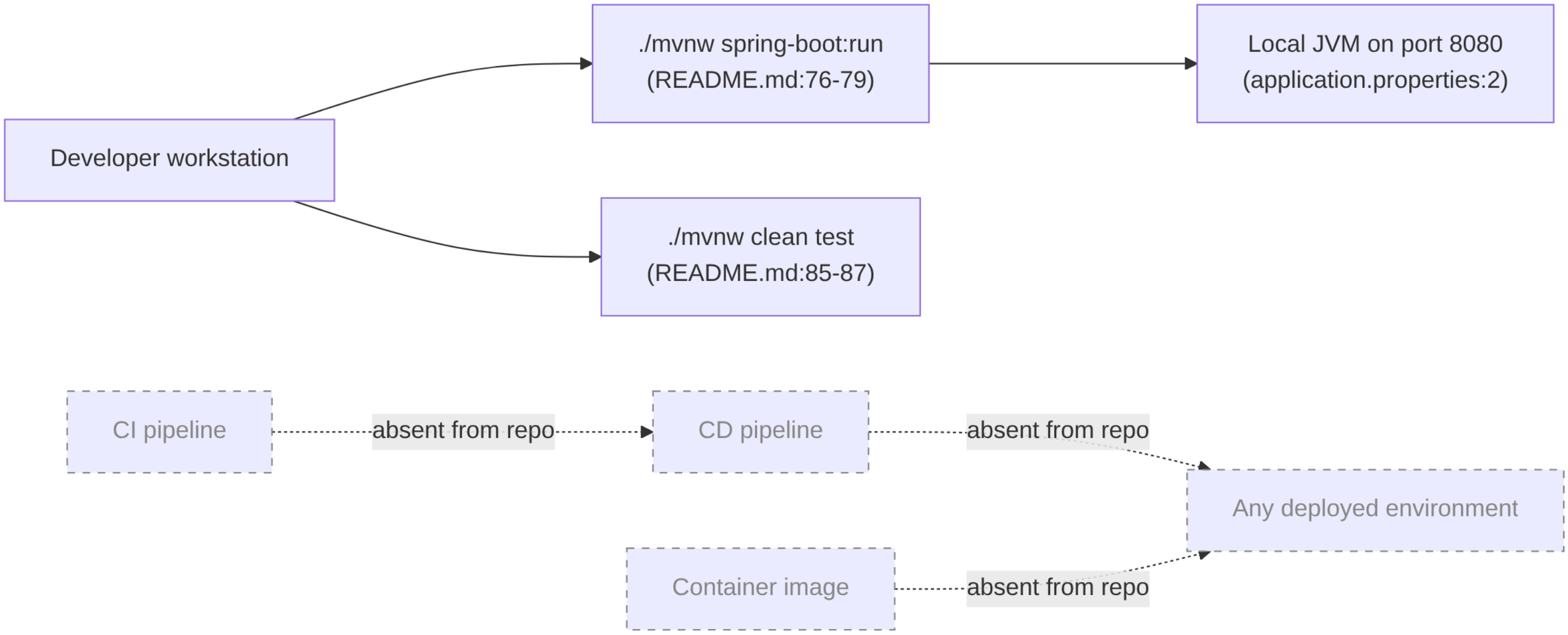
Scope: everything in this document is derived from the files committed on `master` of `COG-GTM/Fixed-Income-RFQ-Trading-Platform` at commit `0b72d81`. Every factual claim carries a `file:line` citation. Anything not read directly from a file is prefixed **Inference:**.

---

1. System context



2. Release flow (as committed)



The repository contains no path to any environment other than a developer laptop. There is no CI workflow directory, no `Dockerfile`, no Kubernetes/Helm/Compose manifest, no infrastructure-as-code and no deployment descriptor anywhere in the tracked file list (`git ls-files` returns 32 files, all of which are source, build wrapper, `README.md`, `LICENSE` and `.gitignore`). The only documented way to run the application is `./mvnw spring-boot:run` (`README.md:76-79`).

---

3. Runtime and framework inventory

Item	Value	Evidence	Support status
Language level	Java 11	`monolith/pom.xml:19`	Java 11 is superseded by 17 and 21; **Inference:** most enterprise baselines now require 17+
Framework	Spring Boot `2.2.6.RELEASE`	`monolith/pom.xml:9`	Out of OSS support (2.2.x reached end of open-source support in 2020); no commercial support purchased that the repo can show
Build tool	Maven via wrapper 0.5.6, Maven 3.6.3	`monolith/.mvn/wrapper/maven-wrapper.properties:1-2`	Maven wrapper `io.takari` 0.5.6 is the pre-`maven-wrapper-plugin` generation
Packaging	Executable JAR, `finalName=monolith`, `spring-boot-maven-plugin`	`monolith/pom.xml:60-67`	Fat JAR — no container image produced by the build
HTTP client stack	Apache HttpComponents `httpclient` 4.x (version managed by the Boot parent)	`monolith/pom.xml:43-46`	HttpClient 4.x is superseded by HttpClient 5
Dev tooling in the runtime classpath	`spring-boot-devtools`, `runtime/optional` scope	`monolith/pom.xml:32-37`	DevTools is a development-only component present in the dependency set

Item	Value	Evidence	Support status
Server port	8080, plaintext HTTP	`monolith/src/main/resources/application.properties:2`	No TLS configuration in the repo
Test stack	JUnit 5 via `spring-boot-starter-test`, vintage engine excluded	`monolith/pom.xml:47-57`	Single test class, 6 tests (`monolith/src/test/java/com/javieraviles/splitthemonolith/IntegrationTest.java:39-168`)

Application entrypoint: `SplitTheMonolithApplication` (`monolith/src/main/java/com/javieraviles/splitthemonolith/SplitTheMonolithApplication.java:25-39`).

---

4. Datastore

Question	Answer	Evidence
Engine	H2, `runtime` scope	`monolith/pom.xml:38-42`
Datasource configuration	**None.** `application.properties` contains only 4 lines and no `spring.datasource.*` key	`monolith/src/main/resources/application.properties:1-4`
Persistence mode	**Inference:** with H2 on the classpath and no datasource URL, Spring Boot auto-configures an embedded in-memory database and Hibernate defaults `ddl-auto` to `create-drop`; all data is therefore lost on process exit. No file in the repo states otherwise.	
Schema definition	Hibernate-generated from the three `@Entity` classes; there is no DDL file, and `ddl-auto` appears nowhere in the repo (grep: zero matches)	`entity/Bond.java:15`, `entity/Counterparty.java:16`, `entity/Rfq.java:20`
Schema migration tool	**None** — zero matches for `flyway` or `liquibase` across the repository	grep, zero-count
ID strategy	`GenerationType.AUTO` on all three entities	`entity/Bond.java:18-20`, `entity/Counterparty.java:19-21`, `entity/Rfq.java:23-25`
Numeric precision	`precision = 19, scale = 2` on all money columns; `precision = 7, scale = 4` on `couponRate`	`entity/Counterparty.java:30,34`, `entity/Bond.java:27,33`, `entity/Rfq.java:38,50`
Seed data	Written on every application start by a `CommandLineRunner`: one counterparty, one bond, one EXECUTED RFQ	`SplitTheMonolithApplication.java:26,41-52`
Open-in-view	Explicitly disabled	`monolith/src/main/resources/application.properties:1`

Data-access style — the load-bearing negative

All persistence goes through three empty Spring Data JPA interfaces with **no declared query methods at all**:

- `repository/BondRepository.java:7`
- `repository/CounterpartyRepository.java:7`
- `repository/RfqRepository.java:7`

Grep zero-counts across `monolith/src`:

Pattern	Occurrences
`@Query`	0
`nativeQuery`	0
`JdbcTemplate`	0
`EntityManager`	0
Stored-procedure calls (`CallableStatement`, `@Procedure`)	0
Vendor-specific SQL strings	0

This is the single most important migration fact in the repository: there is no hand-written SQL to port, so an H2 → PostgreSQL engine change is a configuration, schema-generation and ID-strategy exercise rather than a query-rewriting exercise.

5. Business logic and transaction boundary

`RFQExecutionSaga.executeRfq` (`saga/RFQExecutionSaga.java:29-47`) is annotated `@Transactional` (`saga/RFQExecutionSaga.java:29`) and performs, in one database transaction:

1. load the bond, else 404 (`saga/RFQExecutionSaga.java:32-33`);
2. load the counterparty, else 404 (`saga/RFQExecutionSaga.java:34-35`);
3. `bond.deductNotional(...)` which throws `InsufficientNotionalException` when the requested notional exceeds inventory (`entity/Bond.java:52-57`);
4. `counterparty.deductCredit(...)` which throws `InsufficientCreditException` when the execution price exceeds available credit (`entity/Counterparty.java:58-63`);
5. persist an RFQ with status `EXECUTED` (`saga/RFQExecutionSaga.java:45-46`).

The class is named a saga but the in-code comment states that no compensation is required because everything runs inside a single transaction (`saga/RFQExecutionSaga.java:38-42`). **Migration consequence:** atomicity today depends on both aggregates living in one relational database, so any target that splits counterparty credit and bond inventory into separate datastores turns this into a distributed-transaction problem.

Both exception types map to HTTP 400 via `@ResponseStatus` (`exception/InsufficientNotionalException.java:6`, `exception/InsufficientCreditException.java:6`); `ResourceNotFoundException` maps to 404 (`exception/ResourceNotFoundException.java:6`).

Concurrency

Neither `Counterparty` nor `Bond` carries a `@Version` field (grep for `@Version`: zero matches across `monolith/src`). Credit and notional are read-modify-written inside the transaction (`entity/Counterparty.java:54-63`, `entity/Bond.java:48-57`). **Inference:** with the default read-committed isolation of a server-based RDBMS and more than one application replica, two concurrent RFQs against the same bond or counterparty can lose an update and overdraw inventory or credit. Single-JVM H2 today masks this; horizontal scaling on ECS does not.

---

6. Integrations

Direction	Target	Protocol	Evidence	State
Outbound	`ConfirmationMS` at `${confirmationms.url} + confirmations/`	HTTP POST, JSON, `RestTemplate`	`restclient/TradeConfirmationMicroserviceClient.java:17-26`	Reachable only when the toggle is on
Outbound (configured value)	`http://localhost:8070/`	plaintext HTTP, loopback	`monolith/src/main/resources/application.properties:4`	Hard-coded in the properties file; no service discovery, no DNS name, no TLS
Internal fallback	`TradeConfirmationService.sendTradeConfirmation` — writes one log line	none	`service/TradeConfirmationService.java:14-17`	Default path

The toggle `use.confirmation.service` is `false` (`monolith/src/main/resources/application.properties:3`), bound in `controller/CounterpartyController.java:33-34`, and branched on in `controller/CounterpartyController.java:83-87`. The confirmation is emitted **only on a PATCH that ADDs credit** (`controller/CounterpartyController.java:80-87`) — despite the README's statement that "a trade confirmation is sent to a counterparty whenever credit is added" (`README.md:44`), no confirmation is emitted when an RFQ executes and credit is deducted (`saga/RFQExecutionSaga.java:43`). The README describes the same behaviour the code implements here, but note the confirmation is not part of trade execution at all.

The `RestTemplate` bean is built on a `CloseableHttpClient` with default settings — **no connect timeout, no read timeout, no connection-pool sizing** are configured (`SplitTheMonolithApplication.java:54-64`).

Integration zero-counts

Integration type	Occurrences in `monolith/src`
Messaging (Kafka, JMS, RabbitMQ, SQS/SNS)	0
SMTP / email	0
FTP / SFTP / file shares	0
Scheduled jobs (`@Scheduled`, Quartz)	0
Caching (`@Cacheable`, Redis)	0

Integration type	Occurrences in `monolith/src`
File I/O in application code (`Files.`, `FileInputStream`, `FileWriter`)	0 — the only matches in the repository are inside the Maven wrapper bootstrap `monolith/.mvn/wrapper/MavenWrapperDownloader.java`
`WebClient` / reactive stack	0

Nothing is stateful on local disk, there are no batch windows and there is no broker to migrate. That materially shrinks the migration surface.

---

7. API surface

Seventeen mappings across three controllers, all unauthenticated:

Method	Path	Handler
GET	`/counterparties`	`controller/CounterpartyController.java:45-48`
POST	`/counterparties`	`controller/CounterpartyController.java:50-53`
GET	`/counterparties/{id}`	`controller/CounterpartyController.java:55-59`
PUT	`/counterparties/{id}`	`controller/CounterpartyController.java:61-71`
PATCH	`/counterparties/{id}`	`controller/CounterpartyController.java:73-95`
DELETE	`/counterparties/{id}`	`controller/CounterpartyController.java:97-100`
GET	`/bonds`	`controller/BondController.java:32-35`
POST	`/bonds`	`controller/BondController.java:37-40`
GET	`/bonds/{id}`	`controller/BondController.java:42-46`
PUT	`/bonds/{id}`	`controller/BondController.java:48-59`
PATCH	`/bonds/{id}`	`controller/BondController.java:61-77`
DELETE	`/bonds/{id}`	`controller/BondController.java:79-82`
GET	`/rfqs`	`controller/RfqController.java:31-35`
POST	`/rfqs`	`controller/RfqController.java:37-40`
GET	`/rfqs/{id}`	`controller/RfqController.java:42-46`
DELETE	`/rfqs/{id}`	`controller/RfqController.java:48-51`

Notes carried into the target design:

- `POST /rfqs` accepts the execution price from the client (`dto/RfqDto.java:30-31`, consumed at `saga/RFQExecutionSaga.java:43,46`) — price is not computed server-side.
- The PATCH bodies are untyped `Map<String, String>` (`controller/CounterpartyController.java:74`, `controller/BondController.java:62`); `new BigDecimal(...)` on a missing key raises `NullPointerException`, which is not caught by the `IllegalArgumentException` handler (`controller/CounterpartyController.java:92-94`).
- Bean-validation annotations exist on the DTOs and entities (`dto/RfqDto.java:16,19,22,25,30`, `entity/Counterparty.java:23,29,33`, `entity/Bond.java:32`, `entity/Rfq.java:27,32,37,41,45,49`) but **no controller method parameter is annotated `@Valid`** (grep for `@Valid`: zero matches), so request bodies are not validated at the API boundary.

---

8. Security, identity and configuration

Concern	Finding	Evidence
Authentication / authorisation	**None.** `spring-boot-starter-security` is absent from the POM and there is no `SecurityConfig` (grep: zero matches). Every endpoint above, including `DELETE /counterparties/{id}` and credit-mutating PATCH, is anonymous	`monolith/pom.xml:22-58`, grep zero-count
Session model	Stateless — no `HttpSession` use, no session store	grep zero-count
Transport	Plaintext HTTP on 8080; no TLS/keystore properties	`monolith/src/main/resources/application.properties:2`

Concern	Finding	Evidence
Secrets in the repository	**None found.** The only credential-shaped identifiers are the `MVNW_USERNAME` / `MVNW_PASSWORD` environment variables read by the Maven wrapper for authenticated mirrors — variable names only, no values	`monolith/.mvn/wrapper/MavenWrapperDownloader.java:98-104`, `monolith/mvnw:235-244`
Configuration source	One properties file, four keys, committed to git; no profiles, no environment overrides, no external config service	`monolith/src/main/resources/application.properties:1-4`
CORS	Not configured (grep: zero matches)	grep zero-count
Audit trail	RFQ rows carry only `createdAt`, set in `@PrePersist`; no actor, no update timestamps	`entity/Rfq.java:53-54,69-72`

---

9. Observability

Signal	Finding	Evidence
Health / readiness endpoint	**None** — `spring-boot-actuator` is not a dependency (grep for `actuator`: zero matches). There is no endpoint an ALB target group or ECS health check can poll today	`monolith/pom.xml:22-58`
Metrics	None exported	grep zero-count
Tracing	None	grep zero-count
Logging	SLF4J, one `logger.info` call in the whole application, on the confirmation fallback path; no structured logging or correlation IDs	`service/TradeConfirmationService.java:12,15-16`

---

10. Contradictions and discrepancies found

1. **README claims Java 11 / Spring Boot / H2 and the POM agrees** (`README.md:6-8` vs `monolith/pom.xml:9,19,38-42`) — no contradiction, recorded so the target-state reader does not have to re-check.
2. **Artifact identity does not match the product.** The Maven coordinates, application class and package are still `com.javieraviles / splitthemonolith` with the description "Monolith handling orders, customers and products" (`monolith/pom.xml:12-16`), while the product is a fixed-income RFQ platform (`README.md:1-3`). Any registry/repository naming convention applied at migration time will collide with this.
3. **The "saga" is not a saga** — one `@Transactional` method with no compensation, by its own comment (`saga/RFQExecutionSaga.java:29,38-42`).
4. **The confirmation microservice is configured to `localhost`** (`monolith/src/main/resources/application.properties:4`) and disabled (`:3`), so the integration has never had to work against a real remote address in any committed configuration.
5. **Validation annotations without `@Valid`** — the constraints in `dto/RfqDto.java:16-31` are inert at the boundary.
6. **No health endpoint but a load-balanced target** — every containers-first target design needs a health probe path, and none exists (§9).

---

11. Fact sheet for the target-state design

Dimension	Current state	Evidence
Deployable units	1 fat JAR	`monolith/pom.xml:60-67`
Stateful components	1 embedded database, ephemeral	`monolith/pom.xml:38-42` + absence of datasource config
Local disk state	none	zero-count, §6
Inbound protocols	HTTP/1.1 JSON only	§7
Outbound dependencies	1, disabled	§6
Background processing	none	zero-count, §6
Auth	none	§8
Health probe	none	§9
CI/CD	none in repo	§2
Container image	none in repo	§2

Dimension	Current state	Evidence
IaC	none in repo	§2
Hand-written SQL	none	§4

Target State — AWS

This document proposes the AWS target architecture for the Fixed-Income RFQ Trading Platform. It is constrained by the service allowlist and the platform guardrails in §1 and §2. **Both are assumptions made by this package, not customer policy** — confirming or replacing them is `OQ-01` and `OQ-02` in open-questions.md.

---

1. ASSUMED approved AWS service list

> **ASSUMPTION — not supplied by the customer.** No blessed service list was provided. The list below is the working assumption used to constrain every choice in this document. Anything the customer removes from it invalidates the rows that depend on it; anything they add may simplify them. Confirming this list is **`OQ-01`**, owned by Platform Engineering.

Domain	Assumed-approved services
Compute	Amazon ECS on AWS Fargate
Registry	Amazon ECR
Networking / edge	Amazon VPC (private subnets, NAT), Application Load Balancer, Route 53, AWS Certificate Manager, AWS WAF, VPC endpoints / PrivateLink
Data	Amazon Aurora PostgreSQL (provisioned, Multi-AZ), Amazon S3, AWS Database Migration Service
Security	AWS IAM, AWS KMS, AWS Secrets Manager, AWS Systems Manager Parameter Store, AWS Control Tower, AWS Organizations / SCPs
Observability	Amazon CloudWatch (Logs, Metrics, Alarms), AWS X-Ray, AWS CloudTrail, AWS Config
Delivery	The organisation's approved CI/CD tooling (assumed external to AWS), Terraform for all infrastructure
Resilience	Aurora Global Database, Route 53 health checks and failover routing

Explicitly **assumed NOT on the list** until confirmed: AWS Lambda, Amazon EventBridge, AWS Step Functions, Amazon MQ / Amazon MSK, AWS App Runner, Amazon EKS, AWS AppConfig. Where one of these is the natural fit, the row below is tagged `OFF-LIST?` with an in-list alternative.

2. Platform migration guardrails in force

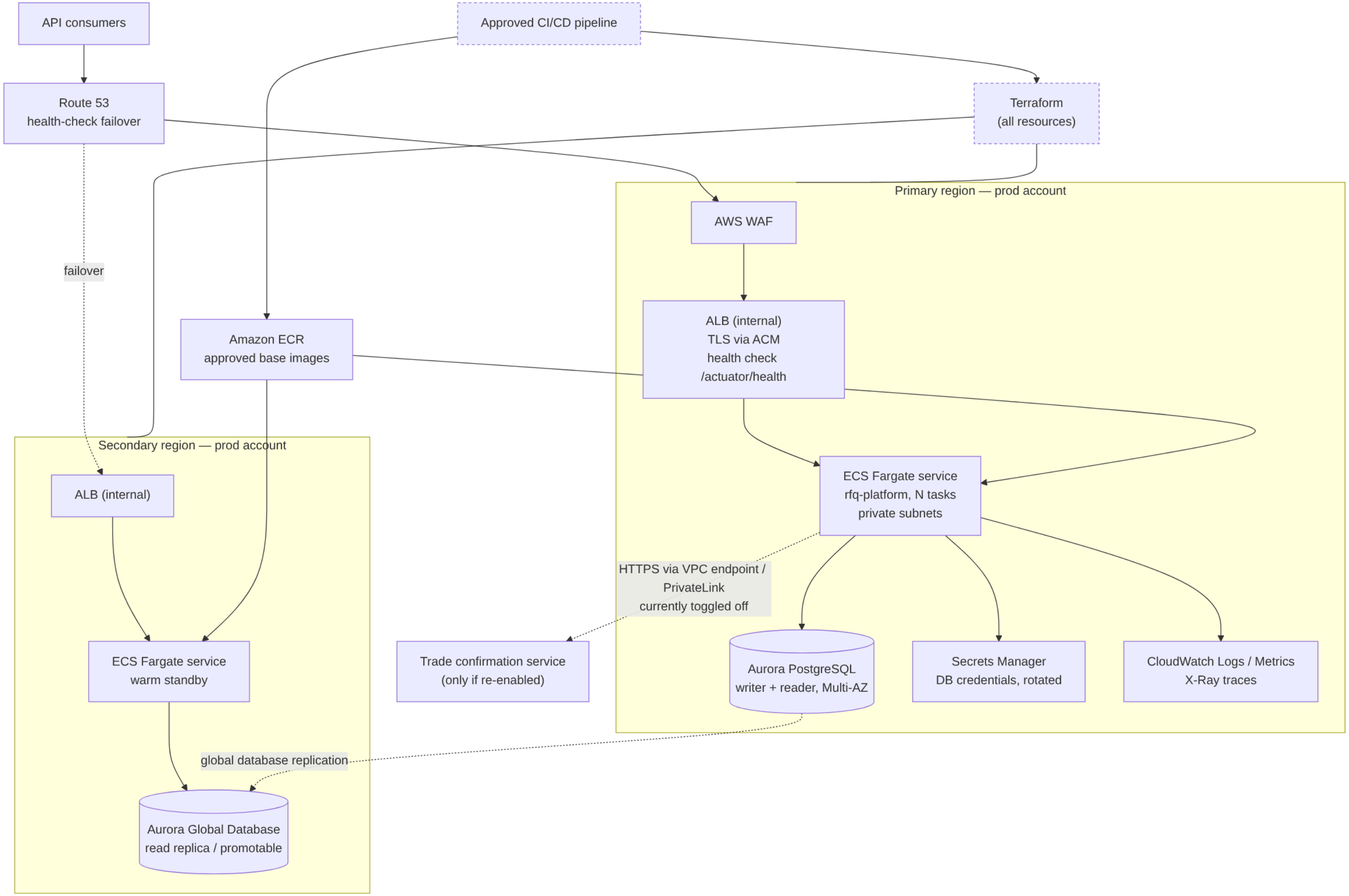
> **ASSUMPTION — these are the playbook's default guardrail set, labelled as assumed defaults.** They are not a statement of the customer's own policy. Confirming or replacing them is **`OQ-02`**, owned by Platform Engineering.

#	Guardrail
G1	**Approved-service allowlist.** Only services on the approved list may appear in the target architecture; the allowlist is assumed to be enforced organisation-wide by service control policies.
G2	**Containers first.** The approved managed container platform is the preferred compute target; VM-based lift-and-shift requires explicit written justification.
G3	**Standard target RDBMS.** Relational workloads target PostgreSQL; remaining on a non-standard engine is an exception that must be justified and flagged.
G4	**Account vending and environment promotion.** Workloads onboard through the standard account-vending process (AWS Control Tower) into separate dev, QA and prod accounts; the dev account must be running before QA is granted, and changes promote dev → QA → prod.
G5	**Everything as code.** All target infrastructure is defined in Terraform; no console-built resources.
G6	**Approved delivery toolchain.** CI/CD runs on the organisation's approved pipeline tooling; moving off any legacy delivery mechanism is part of the plan.
G7	**Production entry criteria.** Prod requires the stated resilience posture (multi-region), load balancing, and a demonstrated DR/failover test before approval.
G8	**Security baseline.** Private networking by default, encryption in transit and at rest, least-privilege IAM roles with no long-lived static credentials, secrets only in the approved secrets manager.
G9	**Approved artifact sources.** Container images and dependencies come only from approved registries/repositories.
G10	**Tagging and observability standards.** All resources carry owner, cost centre, environment and data-classification tags and emit logs/metrics to the central observability platform.

Environment posture used throughout (also an assumption, from the task brief): **multi-AZ in dev, multi-region in prod**; target RDBMS **Aurora PostgreSQL**; target compute **ECS Fargate**; IaC **Terraform**; CD on the org's approved pipeline tooling.

---

3. Target architecture


API consumers
Approved CI/CD pipeline
Route 53
health-check failover
Terraform
(all resources)
failover
Primary region — prod account
AWS WAF
Amazon ECR
approved base images
ALB (internal)
TLS via ACM
health check
/actuator/health
Secondary region — prod account
ALB (internal)
ECS Fargate service
rfq-platform, N tasks
private subnets
HTTPS via VPC endpoint /
PrivateLink
currently toggled off
ECS Fargate service
warm standby
Trade confirmation service
(only if re-enabled)
Aurora PostgreSQL
writer + reader, Multi-AZ
Secrets Manager
DB credentials, rotated
CloudWatch Logs / Metrics
X-Ray traces
global database replication
Aurora Global Database
read replica / promotable

4. Component-by-component mapping

Effort is expressed in Devin-session-equivalents (one session ≈ one to two engineer-weeks of conventional effort). Risk is H/M/L.

#	Current component (evidence)	Target AWS service	Rationale	Risk	Effort
C1	Fat JAR run by `./mvnw spring-boot:run` (`README.md:76-79`, `monolith/pom.xml:60-67`)	**ECS Fargate service** behind an internal ALB	G2 containers-first; no VM justification exists because the app is stateless with no local-disk state (current-state §6)	M	1 session
C2	No container image in the repo (`git ls-files`, current-state §2)	**ECR repository** + a multi-stage image built from an approved base (G9)	An image is the unit of promotion under G4; base image must come from the approved registry	M	0.5 session
C3	H2 in-memory, no datasource config (`monolith/pom.xml:38-42`, `application.properties:1-4`)	**Aurora PostgreSQL**, Multi-AZ in dev/QA, **Aurora Global Database** in prod	G3 standard engine. Zero hand-written SQL (current-state §4) makes this a configuration and schema-generation change, not a query rewrite	M	1.5 sessions
C4	Hibernate-generated schema, `GenerationType.AUTO`, no migration tool (`entity/Bond.java:18-20`, grep zero-count for flyway/liquibase)	**Flyway or Liquibase baseline scripts**, applied by the pipeline; `ddl-auto=validate` in every deployed environment	`AUTO` resolves to identity on H2 and to sequences on PostgreSQL, so the generated DDL differs between the two engines; a versioned baseline removes the ambiguity and satisfies G5	H	1 session
C5	Seed data written on every boot (`SplitTheMonolithApplication.java:26,41-52`)	**Removed from the runtime path**; dev/QA seeding becomes a pipeline task	Against a persistent Aurora cluster this inserts duplicate rows on every task start and will violate the unique ISIN constraint (`entity/Bond.java:22`)	H	0.25 session
C6	No health endpoint (`monolith/pom.xml:22-58`, grep zero-count for actuator)	**Spring Boot Actuator** `/actuator/health` wired to the ALB target group and the ECS container health check	Without it, no target group can determine task health; blocks G7 load balancing	M	0.25 session
C7	Config in a committed properties file (`application.properties:1-4`)	**SSM Parameter Store** for non-secret config, **Secrets Manager** for the DB credential, injected as ECS task-definition secrets	G8: no secrets in code or config; per-environment values without rebuilding the image	L	0.5 session
C8	No authentication on any endpoint (current-state §8)	**ALB internal-only + WAF**, plus an application-level authN/authZ decision (`OQ-14`)	Private networking (G8) reduces exposure but is not authorisation; a trading API that mutates credit cannot ship to prod anonymous	H	1 session (design + implement)
C9	Plaintext HTTP on 8080 (`application.properties:2`)	**TLS terminated at the ALB with ACM**, HTTP only inside the VPC security group	G8 encryption in transit	L	0.25 session
C10	`RestTemplate` on HttpClient 4 with no timeouts to `http://localhost:8070/` (`SplitTheMonolithApplication.java:54-64`, `application.properties:4`)	**HTTPS to a resolvable endpoint via VPC endpoint / PrivateLink**, timeouts and retry budget configured; endpoint value from Parameter Store	A loopback URL cannot survive containerisation; missing timeouts turn a slow dependency into thread-pool exhaustion. Currently disabled (`application.properties:3`), so this is only on the critical path if the customer re-enables it (`OQ-09`)	M	0.5 session
C11	One `logger.info`, no metrics, no tracing (`service/TradeConfirmationService.java:12-16`)	**CloudWatch Logs** via the `awslogs` driver, **CloudWatch metrics**, **X-Ray** traces, structured JSON logging	G10 central observability	M	0.5 session
C12	No CI, no CD (current-state §2)	**Approved CI/CD pipeline**: build → test → image scan → push to ECR → Terraform plan/apply → ECS deploy, promoting dev → QA → prod	G4 and G6	M	1 session
C13	No IaC (current-state §2)	**Terraform** modules for VPC, ALB, ECS, Aurora, IAM, Secrets, CloudWatch, WAF, Route 53	G5	M	1.5 sessions
C14	No accounts or landing zone	**Control Tower–vended dev, QA and prod accounts**, dev first	G4 explicitly sequences dev before QA	L	0.5 session (mostly customer process time)
C15	Java 11 / Spring Boot 2.2.6 (`monolith/pom.xml:9,19`)	**Java 17 or 21 on Spring Boot 3.x**, on an approved base image	Boot 2.2.x is out of open-source support; approved base images are unlikely to ship a Java 11 runtime (G9). Carries the `javax.*` → `jakarta.*` namespace change across all entities and DTOs (`entity/Bond.java:6-11`, `dto/RfqDto.java:6-7`)	H	2 sessions
C16	`spring-boot-devtools` in the dependency set (`monolith/pom.xml:32-37`)	**Excluded from the production image**	DevTools should not be present in a deployed artifact	L	0.1 session
C17	No `@Version`, read-modify-write on credit and notional (`entity/Counterparty.java:54-63`, `entity/Bond.java:48-57`)	**Optimistic locking (`@Version`) or `SELECT … FOR UPDATE`** before running more than one task	Single-JVM H2 hides the race; N Fargate tasks against one Aurora writer do not. This is a correctness blocker on the cutover gate	H	0.5 session

#	Current component (evidence)	Target AWS service	Rationale	Risk	Effort
C18	Artifact coordinates still `com.javieraviles / splitthemonolith` (`monolith/pom.xml:12-16`)	**Renamed image, repository and resource names** to the platform's naming standard	Required for G10 tagging/ownership and for anyone to find the workload	L	0.25 session
C19	Client-supplied execution price (`dto/RfqDto.java:30-31`, `saga/RFQExecutionSaga.java:43`)	**Unchanged by the migration**; recorded as a product decision, not an infrastructure one	Out of scope for lift-to-AWS, but a reviewer will ask (`OQ-15`)	—	—
C20	Saga atomicity in one DB transaction (`saga/RFQExecutionSaga.java:29,38-42`)	**One Aurora cluster, one transaction — unchanged**	Preserving a single relational transaction avoids inventing a distributed transaction; any future service split reopens this	L	0

Off-list temptations, and the in-list alternative used instead

Natural choice	Status	In-list alternative adopted	Why
Amazon EventBridge for asynchronous trade confirmations	`OFF-LIST?`	Synchronous HTTPS call from the ECS task through a VPC endpoint (C10)	Keeps the existing call shape; no new integration pattern to approve
AWS Step Functions to orchestrate the "saga"	`OFF-LIST?`	Keep the single `@Transactional` method inside the service (C20)	There is no compensation logic to orchestrate (`saga/RFQExecutionSaga.java:38-42`)
AWS Lambda for the seed/bootstrap job	`OFF-LIST?`	A one-shot ECS task run by the pipeline (C5)	Reuses the same image and the same execution role
Amazon EKS	`OFF-LIST?` (assumed)	ECS Fargate (C1)	One service, no platform team to run a cluster; ECS Fargate satisfies G2
AWS AppConfig for the `use.confirmation.service` toggle	`OFF-LIST?`	SSM Parameter Store value read at task start (C7)	The toggle changes rarely; a task-definition redeploy is acceptable
Amazon RDS Proxy	`OFF-LIST?`	Application-side HikariCP pool sizing (C1)	Only needed if task counts grow; revisit with `OQ-07`

---

5. Guardrail compliance

Guardrail	How the target design satisfies it	Exception raised
G1 approved-service allowlist	Every service in §3 comes from the §1 list; six natural choices were rejected and tagged `OFF-LIST?` with in-list alternatives (§4)	The list itself is assumed — `OQ-01`
G2 containers first	ECS Fargate (C1) with an ECR image (C2); no EC2 instance appears anywhere in the design, so no lift-and-shift justification is needed	None
G3 standard target RDBMS	Aurora PostgreSQL (C3); H2 is not retained in any environment, including tests, after WS3	None
G4 account vending and promotion	Control Tower–vended dev, QA and prod accounts (C14); pipeline promotes dev → QA → prod (C12); WS1 delivers the dev account before QA is requested	None
G5 everything as code	Terraform modules for every resource in §3 (C13); the only manual step is the account-vending request itself	None
G6 approved delivery toolchain	The pipeline in C12 replaces the current "developer laptop" release path (current-state §2) — there is no legacy pipeline to retire, only an absence to fill	None
G7 production entry criteria	Multi-region prod with Aurora Global Database and Route 53 failover (§3); ALB load balancing (C1, C9); DR failover test is an explicit entry criterion of WS9 in the migration plan	Multi-region prod is an assumption from the brief — `OQ-03`
G8 security baseline	Private subnets and internal ALB (C8); TLS at the ALB with ACM (C9); Aurora encrypted with KMS; DB credential in Secrets Manager with rotation (C7); ECS task and execution roles scoped per environment, no static keys. The repository contains no committed secrets today (current-state §8)	The application has no authN/authZ at all (C8) — a prod blocker, `OQ-14`
G9 approved artifact sources	Image built `FROM` an approved base and pushed to ECR (C2); Maven resolves through the approved internal repository, which the wrapper already supports via `MVNW_REPOURL` (`monolith/mvnw:214-215`)	Approved base image must ship Java 17/21, forcing C15 — `OQ-05`
G10 tagging and observability	Terraform applies the mandatory tag set to every resource; Actuator health (C6), CloudWatch logs/metrics and X-Ray (C11)	Data classification for counterparty credit and LEI data is undetermined — `OQ-12`

---

6. What the target explicitly does not change

- The domain model and the three tables it maps to (`entity/Bond.java:15`, `entity/Counterparty.java:16`, `entity/Rfq.java:20`).
- The public REST contract (current-state §7) — paths, verbs and payloads are preserved so that consumers are unaffected by the platform move.
- The single-transaction execution semantics (`saga/RFQExecutionSaga.java:29`).
- Any decomposition into microservices. The confirmation client (`restclient/TradeConfirmationMicroserviceClient.java:23-26`) shows an earlier decomposition intent, but splitting services during a platform migration would change two variables at once.

Open Questions

Every question below is a fact the repository cannot settle. Each has an ID referenced by migration-plan.md, and each names the role that owns the answer. Questions `OQ-01` and `OQ-02` gate the whole design: the service allowlist and the guardrail set used in target-state.md are **assumptions made by this package**, not customer policy.

Platform Engineering

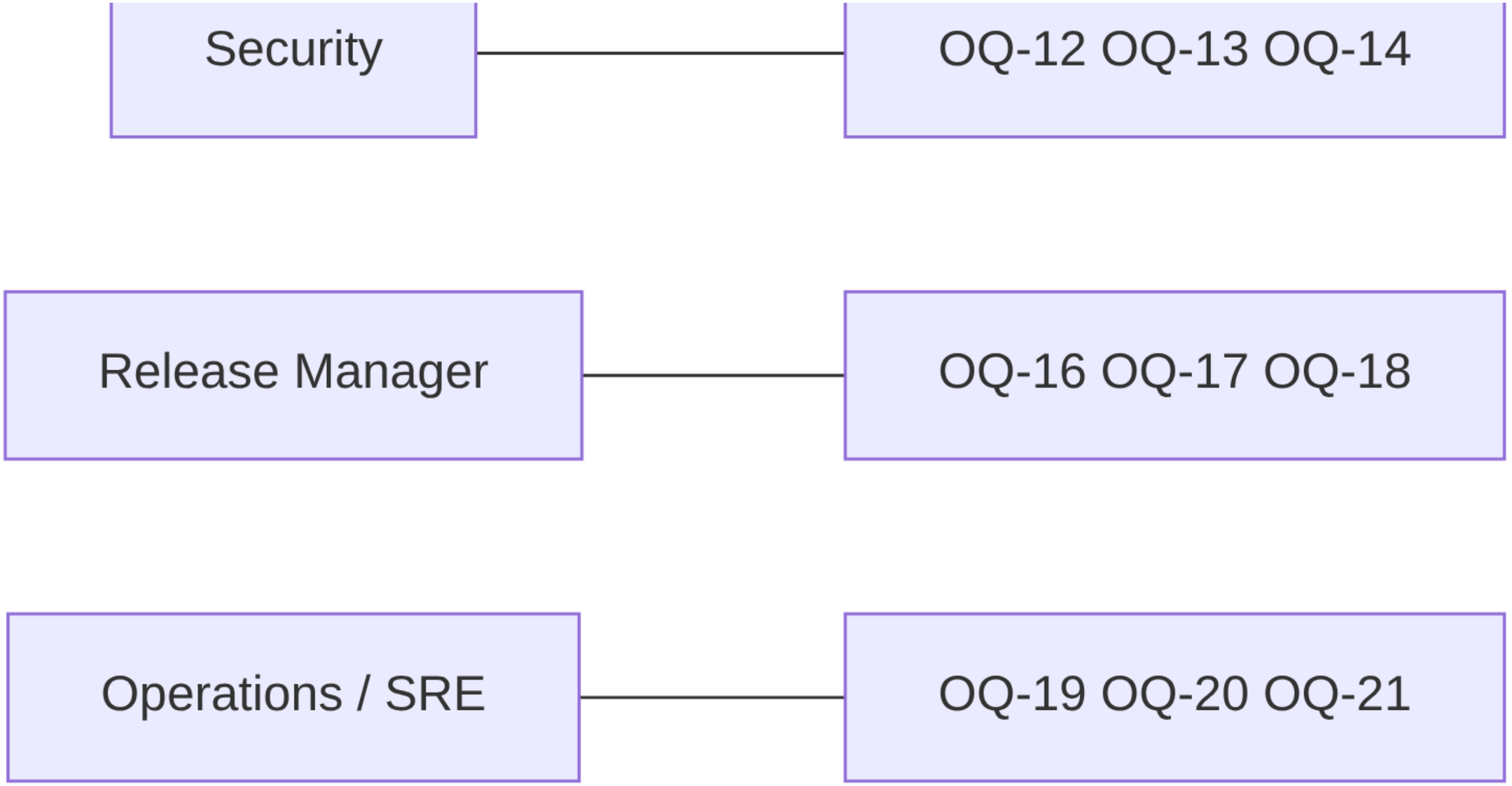
OQ-01 OQ-02 OQ-03 OQ-04 OQ-05 OQ-06

DBA

OQ-07 OQ-08

Application Owner

OQ-09 OQ-10 OQ-11 OQ-15



Programme and platform

ID	Question	Owner	Why it matters	Blocks
OQ-01	The approved ("blessed") AWS service list was not supplied. Is the assumed list in target-state.md §1 correct, and specifically: are Lambda, EventBridge, Step Functions, EKS, App Runner, RDS Proxy and AppConfig genuinely off-list?	Platform Engineering	Every `OFF-LIST?` row in the target state exists only because of this assumption; approving any of them simplifies the design	WS0, and the sign-off of the whole target state
OQ-02	No customer platform guardrails were supplied, so the playbook's default set (G1–G10) is in force as an assumption. Are these your guardrails, and what replaces or supplements them?	Platform Engineering	The guardrail-compliance table is only meaningful against the real policy	WS0
OQ-03	Is multi-region prod actually required for this workload, or is Multi-AZ with a documented RPO sufficient? The brief states multi-region; nothing in the repo justifies it	Platform Engineering	Aurora Global Database and a second-region ECS service roughly double the prod infrastructure cost and add a failover-test obligation to WS9	WS8, WS9
OQ-04	Which AWS Organizations OU should the dev, QA and prod accounts be vended into, and what is the lead time for the Control Tower request?	Platform Engineering	G4 sequences dev before QA; account lead time is usually the longest external wait in the plan	WS1

ID	Question	Owner	Why it matters	Blocks
OQ-05	Which approved container base images are available, and which Java runtime versions do they carry?	Platform Engineering	If no approved base ships Java 11, the Spring Boot 3 / Java 17+ upgrade (C15) moves from "recommended" to "prerequisite" and onto the critical path	WS2, WS4
OQ-06	Which Terraform module registry, state backend and provider version constraints are mandated?	Platform Engineering	Determines whether WS2 writes modules or consumes existing ones	WS2

Database

ID	Question	Owner	Why it matters	Blocks
OQ-07	What are the real production data volumes and transaction rates? The repository contains only three seeded rows (`SplitTheMonolithApplication.java:41-52`)	DBA	Sets the Aurora instance class, connection-pool sizing and whether RDS Proxy needs to be added to the allowlist	WS3
OQ-08	Is there an existing production database for this platform, and if so on which engine — because the committed configuration is an ephemeral in-memory H2 with no datasource URL (`monolith/src/main/resources/application.properties:1-4`)? If one exists, DMS-based data migration and a cutover freeze window are needed; if not, WS3 is a greenfield schema	DBA	This single answer changes WS3 from "create schema" to "migrate data with DMS, validate row counts, plan a freeze"	WS3, WS9

Application

ID	Question	Owner	Why it matters	Blocks
OQ-09	Will the trade-confirmation integration be re-enabled on AWS? It is toggled off (`monolith/src/main/resources/application.properties:3`) and points at `http://localhost:8070/` (`:4`)	Application Owner	If it stays off, C10 (endpoint, TLS, timeouts, PrivateLink) drops out of scope; if it is turned on, the target service's real address, protocol and SLA are needed	WS5
OQ-10	Is this repository the artefact that is actually deployed today, and is any other component (UI, gateway, scheduler) in front of it? The repo contains no client and no gateway configuration	Application Owner	A hidden consumer would constrain the "REST contract unchanged" commitment in target-state.md §6	WS0
OQ-11	Who are the API consumers, and can they tolerate a DNS/hostname change at cutover?	Application Owner	Determines whether Route 53 must serve the legacy hostname and whether a dual-run period is needed	WS9

ID	Question	Owner	Why it matters	Blocks
OQ-15	`POST /rfqs` accepts the execution price from the client (`dto/RfqDto.java:30-31`, used at `saga/RFQExecutionSaga.java:43,46`). Is client-supplied pricing intentional?	Application Owner	Not a migration item, but a reviewer will raise it; if it is a defect it should be tracked outside this docs-only package	—

Security

ID	Question	Owner	Why it matters	Blocks
OQ-12	What is the data classification of counterparty names, LEIs, credit limits and executed trade records?	Security	Drives KMS key policy, log-redaction rules, the mandatory data-classification tag (G10) and whether CloudWatch Logs can hold request payloads	WS2, WS6
OQ-13	Is an internal-only ALB acceptable, or is external exposure required — and if external, is WAF plus a managed rule set sufficient?	Security	Determines the VPC/edge design and whether public subnets appear at all	WS2
OQ-14	The application has no authentication or authorisation of any kind: every endpoint, including `DELETE /counterparties/{id}` (`controller/CounterpartyController.java:97-100`) and credit-mutating `PATCH` (`controller/CounterpartyController.java:73-95`), is anonymous. What authN/authZ must be in place before production?	Security	This is a hard prod-cutover blocker in WS9; network isolation alone is not authorisation	WS7, WS9

Delivery

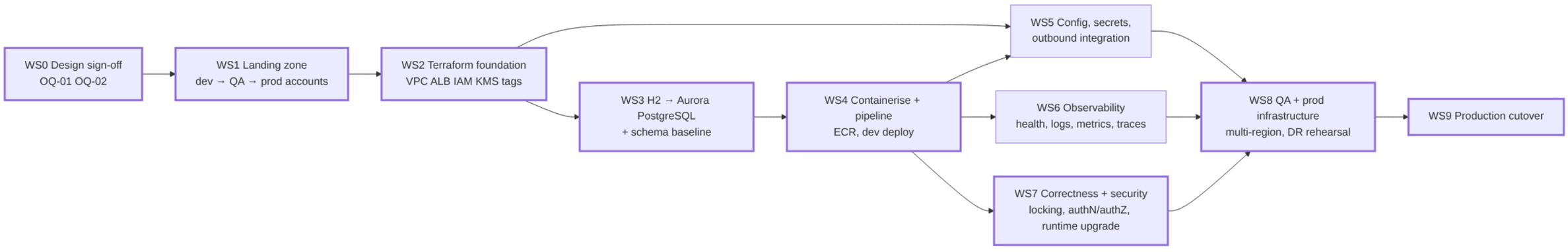
ID	Question	Owner	Why it matters	Blocks
OQ-16	Which pipeline tooling is approved, and does a reference pipeline for "container to ECS Fargate via Terraform" already exist?	Release Manager	Determines whether WS4 is configuration or construction; the repo has no CI or CD at all today	WS4
OQ-17	What are the change-approval and release-window rules for a production deployment in this organisation?	Release Manager	Sets the WS9 cutover date and whether a rollback rehearsal is mandatory	WS9
OQ-18	Which internal Maven repository must builds resolve through, and what credential mechanism does the pipeline use? The wrapper already supports `MVNW_REPOURL` and `MVNW_USERNAME`/`MVNW_PASSWORD` (`monolith/mvnw:214-215,235-244`)	Release Manager	G9 approved artifact sources; a build that reaches out to Maven Central directly will fail policy	WS4

Operations

ID	Question	Owner	Why it matters	Blocks
OQ-19	What are the RTO and RPO for this platform?	Operations / SRE	Decides Aurora backup retention, whether Global Database is warranted (with `OQ-03`) and what the WS9 DR test must demonstrate	WS8, WS9
OQ-20	Which central observability platform must receive logs and metrics, and is CloudWatch the destination or only a hop?	Operations / SRE	G10; changes the log-driver and agent configuration in the task definition	WS6
OQ-21	Who is the on-call owner and what is the cost centre for the mandatory tag set?	Operations / SRE	G10 tagging cannot be implemented in Terraform without concrete values	WS2

Migration Plan

Ten workstreams, each independently reviewable and independently deliverable. Effort is in Devin-session-equivalents (one session ≈ one to two engineer-weeks of conventional effort); elapsed time is dominated by the external waits called out per workstream, not by the engineering. Question IDs refer to open-questions.md; component IDs (C1…C20) refer to target-state.md §4.



Critical path: WS0 → WS1 → WS2 → WS3 → WS4 → WS7 → WS8 → WS9. WS5 and WS6 run in parallel with WS7 and only need to land before WS8. The longest external waits are the Control Tower account vending in WS1 (`OQ-04`) and the change-approval window in WS9 (`OQ-17`).

WS0 — Design sign-off and input capture

Entry criteria: this package reviewed by Platform Engineering, Security, the DBA and the Application Owner.

Work: replace the assumed service allowlist and the assumed guardrail set with the customer's own (`OQ-01`, `OQ-02`); confirm the resilience posture (`OQ-03`); confirm this repository is the deployed artefact and identify any consumer in front of it (`OQ-10`).

Exit criteria: allowlist and guardrails confirmed in writing; every `OFF-LIST?` row in target-state.md §4 either accepted with its in-list alternative or reopened.

Effort: 0.25 session plus review time. **Dependencies:** none.

WS1 — Landing zone and account vending

Entry criteria: WS0 complete; OU placement and lead time known (`OQ-04`).

Work: raise the Control Tower account-vending request for **dev first**, as G4 requires, then QA, then prod. Establish the Terraform state backend and provider constraints (`OQ-06`). Capture the mandatory tag values — owner, cost centre, environment, data classification (`OQ-21`, `OQ-12`).

Exit criteria: dev account exists and is reachable by the pipeline identity; QA request is queued but not required yet.

Effort: 0.5 session of engineering; elapsed time is the vending lead time. **Dependencies:** WS0.

WS2 — Terraform foundation

Entry criteria: dev account available; base-image catalogue known (`OQ-05`); exposure model decided (`OQ-13`).

Work (C13, C9, C7, C2): Terraform modules for VPC with private subnets and NAT, VPC endpoints, internal ALB with an ACM certificate, WAF, ECR repository, KMS keys, IAM task and execution roles, Secrets Manager and Parameter Store entries, CloudWatch log groups, and the mandatory tag set applied to everything. No console-built resources (G5).

Exit criteria: `terraform apply` produces an empty-but-complete dev environment; a policy scan shows no public endpoints and no static credentials.

Effort: 1.5 sessions. **Dependencies:** WS1.

WS3 — H2 → Aurora PostgreSQL

Entry criteria: answers to `OQ-07` (volumes) and, critically, `OQ-08` (does a real production database exist at all — the committed configuration is an ephemeral in-memory H2 with no datasource URL, `monolith/src/main/resources/application.properties:1-4`).

Work (C3, C4, C5):

1. Provision Aurora PostgreSQL in dev, Multi-AZ, KMS-encrypted, credential in Secrets Manager.
2. Introduce a versioned schema baseline (Flyway or Liquibase) generated from the three entities, and set `ddl-auto=validate` in every deployed environment. `GenerationType.AUTO` (`entity/Bond.java:18-20`, `entity/Counterparty.java:19-21`, `entity/Rfq.java:23-25`) resolves differently on PostgreSQL than on H2, so the baseline must pin the sequence strategy explicitly.
3. Remove the boot-time seeding `CommandLineRunner` (`SplitTheMonolithApplication.java:26,41-52`) from the runtime path — against a persistent cluster it re-inserts on every task start and violates the unique ISIN constraint (`entity/Bond.java:22`) — and move seeding to a pipeline task.
4. Point the test suite at PostgreSQL (Testcontainers or a dev instance) so the six existing tests (`monolith/src/test/java/com/javieraviles/splitthemonolith/IntegrationTest.java:39-168`) exercise the target engine.
5. Only if `OQ-08` reveals a real source database: DMS full-load plus CDC, with row-count and checksum validation.

Why this is a workstream and not a table row: the engine change touches ID generation, schema ownership, test infrastructure and — if data exists — a freeze window. What makes it *tractable* is that the application contains **zero hand-written SQL, zero native queries and zero stored procedures** (current-state §4), so no query rewriting is required.

Exit criteria: the full test suite passes against Aurora PostgreSQL; schema is created only by the migration tool.

Effort: 1.5 sessions, plus 1 session if DMS is in scope. **Dependencies:** WS2.

WS4 — Containerisation and delivery pipeline

Entry criteria: approved base image identified (`OQ-05`); pipeline tooling and reference pipeline known (`OQ-16`); internal Maven repository and credential mechanism known (`OQ-18`).

Work (C1, C2, C12, C16, C18): multi-stage Dockerfile from an approved base, excluding `spring-boot-devtools` (`monolith/pom.xml:32-37`) from the runtime image; rename artefact, image and resource identifiers off `com.javieraviles / splitthemonolith` (`monolith/pom.xml:12-16`); pipeline stages build → test → image scan → push to ECR → `terraform plan/apply` → ECS service deploy; ECS Fargate service and task definition in dev.

Exit criteria: a commit on the default branch deploys automatically to dev and the seeded endpoints respond through the internal ALB.

Effort: 1.5 sessions. **Dependencies:** WS3 (the task needs a database to start against).

WS5 — Configuration, secrets and the outbound integration

Entry criteria: WS4 deploying to dev; `OQ-09` answered (is trade confirmation re-enabled?).

Work (C7, C10): move all four properties (`monolith/src/main/resources/application.properties:1-4`) to Parameter Store and Secrets Manager, injected as task-definition secrets; if confirmation is re-enabled, replace `http://localhost:8070/` with a resolvable HTTPS endpoint reached through a VPC endpoint or PrivateLink and add connect/read timeouts and a retry budget to the `RestTemplate` (`SplitTheMonolithApplication.java:54-64`), which currently has none.

Exit criteria: no environment-specific value is baked into the image; if enabled, the confirmation call succeeds against the real endpoint with timeouts proven by a fault-injection test.

Effort: 0.5 session (1 session if the integration is re-enabled). **Dependencies:** WS2, WS4.

WS6 — Observability

Entry criteria: WS4 deploying to dev; central observability destination known (`OQ-20`); data classification known so logs can be redacted correctly (`OQ-12`).

Work (C6, C11): add Spring Boot Actuator and wire `/actuator/health` to both the ALB target group and the ECS container health check — there is no health endpoint today (current-state §9); structured JSON logging with correlation IDs to replace the single `logger.info` (`service/TradeConfirmationService.java:15-16`); CloudWatch metrics and alarms on 5xx rate, task restarts, Aurora CPU and connection saturation; X-Ray tracing.

Exit criteria: a deliberately failed task is detected and replaced by the health check; alarms fire into the central platform.

Effort: 0.5 session. **Dependencies:** WS4.

WS7 — Correctness and security blockers

Entry criteria: WS4 deploying to dev; `OQ-14` answered (required authN/authZ model).

Work:

1. **Concurrency (C17).** Add `@Version` optimistic locking, or pessimistic `SELECT … FOR UPDATE`, to `Counterparty` and `Bond`. Credit and notional are read-modify-written (`entity/Counterparty.java:54-63`, `entity/Bond.java:48-57`) with no version column anywhere in the model; a single H2-backed JVM hides the race, multiple Fargate tasks against one Aurora writer do not.
2. **Authentication and authorisation (C8).** The API is entirely anonymous today (current-state §8), including `DELETE /counterparties/{id}` (`controller/CounterpartyController.java:97-100`) and the credit-mutating `PATCH` (`controller/CounterpartyController.java:73-95`).
3. **Input validation.** Apply `@Valid` at the controller boundary so the existing constraints (`dto/RfqDto.java:16-31`) are enforced, and handle the missing-key `NullPointerException` path in the PATCH handlers (`controller/CounterpartyController.java:74-94`, `controller/BondController.java:62-76`).
4. **Runtime upgrade (C15).** Java 17/21 on Spring Boot 3.x, including the `javax.*` → `jakarta.*` namespace change across entities and DTOs (`entity/Bond.java:6-11`, `dto/RfqDto.java:6-7`) and the HttpClient 4 → 5 change implied by `monolith/pom.xml:43-46`. Sequencing note: if no approved base image ships Java 11 (`OQ-05`), this item moves ahead of WS4 and onto the critical path.

These changes are deliberately out of the docs-only PR that carries this package. They are tracked here because WS9 is gated on them.

Exit criteria: a concurrent-execution test proves no lost update; every endpoint requires an authenticated principal; the application runs on a supported Spring Boot line.

Effort: 3 sessions. **Dependencies:** WS4.

WS8 — QA and production infrastructure, and the DR rehearsal

Entry criteria: WS5, WS6 and WS7 complete in dev; QA account vended (G4 requires dev to be running first); `OQ-03` and `OQ-19` answered (resilience posture, RTO/RPO).

Work: promote the Terraform stack into QA and then prod; Aurora Global Database with a secondary region and a warm-standby ECS service; Route 53 health-check failover; backup retention set to the agreed RPO; **execute a documented regional failover test and record the measured RTO and RPO**.

Exit criteria: QA runs the same image and the same Terraform as dev; prod infrastructure exists with no traffic on it; the failover test report is signed off. This satisfies guardrail G7.

Effort: 2 sessions. **Dependencies:** WS5, WS6, WS7.

WS9 — Production cutover *(final workstream)*

Entry criteria — all must be true before any production traffic is served:

#	Criterion	Source
1	Optimistic or pessimistic locking in place and proven under a concurrent-execution test	WS7.1, C17
2	Authentication and authorisation enforced on every endpoint, per the Security answer to `OQ-14`	WS7.2, C8
3	Application running on a supported Spring Boot / Java line	WS7.4, C15
4	Boot-time seeding removed from the runtime path	WS3.3, C5
5	Schema created and versioned only by the migration tool, `ddl-auto=validate` in prod	WS3.2, C4
6	Guardrail G7 met: multi-region prod, load balancing in place, **and the DR failover test executed and signed off** with the measured RTO/RPO against `OQ-19`	WS8
7	Guardrail G8 met: no public endpoints, TLS at the ALB, KMS encryption at rest, DB credential in Secrets Manager with rotation enabled, no static credentials in any task role	WS2, WS5
8	Guardrail G10 met: mandatory tags on every resource, health checks live, alarms firing into the central platform	WS2, WS6
9	Data migration validated by row count and checksum, or `OQ-08` confirms there is no source data to migrate	WS3.5
10	Consumer cutover agreed: DNS strategy and any dual-run period settled with the Application Owner (`OQ-11`)	WS0, `OQ-11`
11	Change approval obtained for the agreed release window (`OQ-17`), with a rehearsed rollback	`OQ-17`

Work: freeze writes on the source system if one exists; final DMS CDC catch-up and validation; switch Route 53 to the AWS ALB; monitor error rate, latency and Aurora saturation through one full trading day; keep the rollback path warm until the agreed soak period ends.

Exit criteria: production traffic served from AWS; rollback path formally retired; the legacy runtime decommissioned.

Effort: 1 session plus the soak period. **Dependencies:** WS8.

Summary

WS	Name	Effort (sessions)	On critical path
WS0	Design sign-off	0.25	yes
WS1	Landing zone	0.5	yes
WS2	Terraform foundation	1.5	yes
WS3	H2 → Aurora PostgreSQL	1.5 – 2.5	yes
WS4	Containerisation and pipeline	1.5	yes
WS5	Config, secrets, integration	0.5 – 1	no
WS6	Observability	0.5	no
WS7	Correctness and security	3	yes
WS8	QA/prod infra and DR rehearsal	2	yes
WS9	Production cutover	1	yes

Total engineering: roughly 12–14 sessions. Elapsed time is set by account vending (`OQ-04`), the answers to the open questions, and the production change window (`OQ-17`) — not by the engineering.